# <알고리즘 과제> 퀵 정렬

**문제: 심층문제 8-3 퀵 정렬의 다양한 버전 비교실험**

**프로그램 요구사항:** 비교 결과가 최종적으로 다음과 같이 인쇄되어야 한다
(**참고:** X.XXXXXXXX는 측정된 cputime in milliseconds).

| **Limit** | 결정적1 | 결정적3 | 무작위1 | 무작위3 |
|---|---|---|---|---|
| 1 | X.XXXXXXXX | X.XXXXXXXX | X.XXXXXXXX | X.XXXXXXXX |
| 100 | X.XXXXXXXX | X.XXXXXXXX | X.XXXXXXXX | X.XXXXXXXX |
| 500 | X.XXXXXXXX | X.XXXXXXXX | X.XXXXXXXX | X.XXXXXXXX |
| 1000 | X.XXXXXXXX | X.XXXXXXXX | X.XXXXXXXX | X.XXXXXXXX |

**힌트:** 주요 함수들의 설계 가이드라인

global integer **n** = 100000
global array **Limits** = {1, 100, 500, 1000}

```
Alg main()
1. A ← createArray()
2. for each Limit in Limits                    {quick-sort limit sizes}
       for each mode in ('deterministic1', 'deterministic3', 'randomized1', 'randomized3')
           A' ←  A                             {copy array}
           quickSort(A', n)
           printCPUTime()
3. return
```

```
Alg createArray()
input none
output array A of size n

1. for i ← 0 to n – 1
       A[i] ← a random integer between 1 and n
2. return A
```

```
Alg quickSort(A, n)                    {deterministic quick sort upto Limit}
input array A, integer n, Limit        {Limit is global}
output sorted array A
```

1. **rQuickSort**(**A**, 0, **n** - 1)
2. if (**Limit** > 1)
   **insertionSort**(**A**, **n**)
3. return

Alg **rQuickSort**(**A**, **l**, **r**) {recursive}
input array **A**, index **l**, **r**, integer **Limit** {**Limit** is global}
output completely or partially sorted array **A**

1. if (**r** - **l** ≥ **Limit**)
   **k** ← **findPivot**(**A**, **l**, **r**)
   **a**, **b** ← **inPlacePartition**(**A**, **l**, **r**, **k**)
   **rQuickSort**(**A**, **l**, **a** - 1)
   **rQuickSort**(**A**, **b** + 1, **r**)

Alg **findPivot**(**A**, **l**, **r**) {find the location of pivot}
1. if (**mode** = **'deterministic1'**)
   return **r**
2. if (**mode** = **'randomized1'**)
   return a random integer between **l** and **r**
3. if (**r** - **l** = **1**)
   return **l**
4. switch (**mode**) {select three positions}
   **'deterministic3'**: **a**, **b**, **c** ← **l**, (**l** + **r**)/2, **r**
   **'randomized3'**: **a**, **b**, **c** ← 3 random integers between **l** and **r**
5. return **findIndexOfMedianOfThree**(**a**, **b**, **c**) {index of median of **A**[**a**], **A**[**b**], **A**[**c**]}

† **주의:** 위 **inPlacePartition**은 부리스트에 중복 키가 존재하더라도 분할을 수행하는 버전이어야 한다(심층문제 **8-2** 참고).